

CS & IT ENGINEERING

Programming in C

Arrays and Pointers

Lec- 01



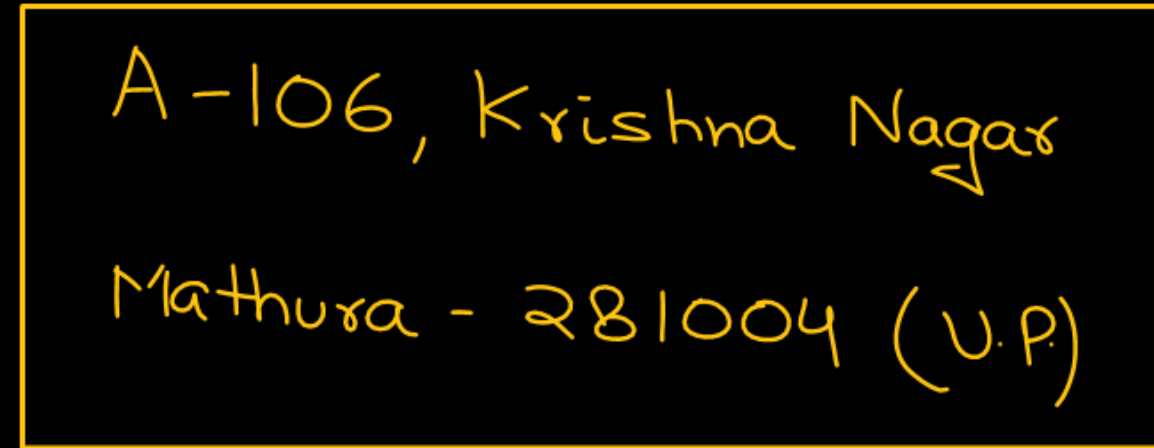
By- Pankaj Sharma sir



TOPICS TO BE
COVERED

Arrays

1) Address



2.)

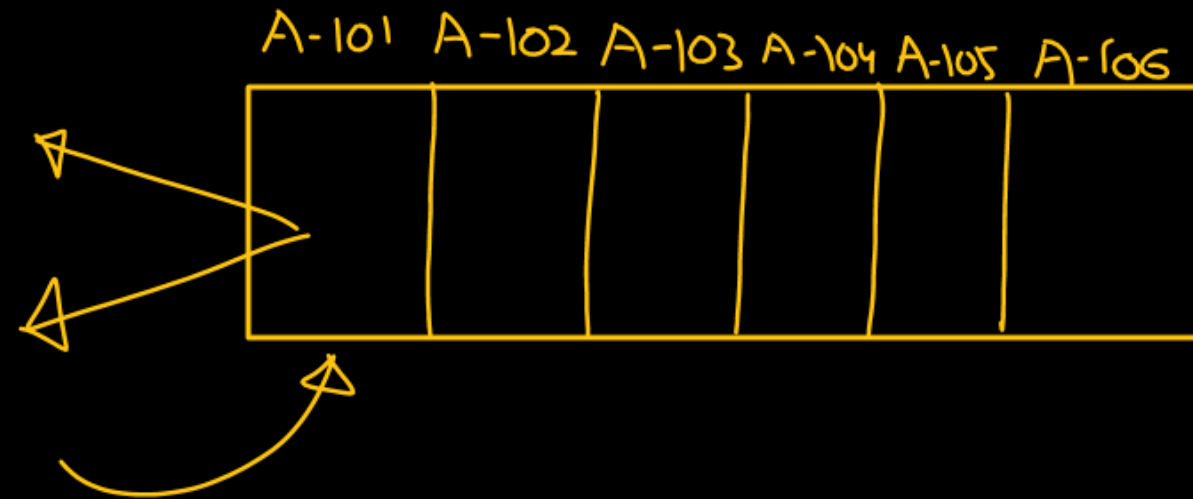
Address $\rightarrow$ Abs.
 $\rightarrow$ relative

Relative Addressing

A-106

① A-106, —

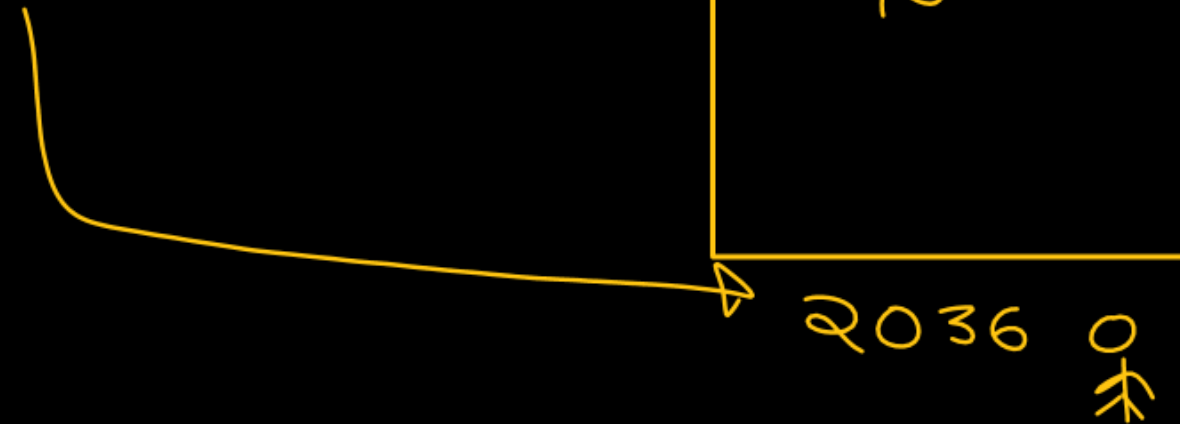
② Relative address



③ How to find address. ($\&$: address of)

int a = 10;

$\&a$:



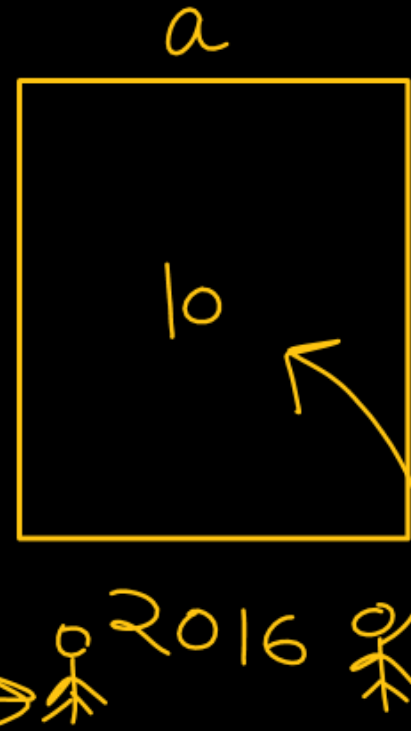
④

* : value at operator

* (Memory location)

int a = 10;

&a : (Memory location 2016)

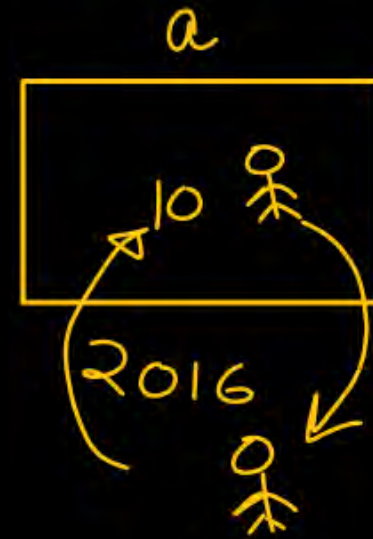


* (&a) $\Rightarrow$ value at (Memory location 2016)

&a $\equiv$ 2016
* (&a) = value at (Memory location 2016)
 $\Rightarrow$ 10

int a = 10;

$\&a \Rightarrow$ Memory
location
2016



Enemies

~~$\&\&a = a$~~

(i) $a \Rightarrow 10$ (inside)

(ii) $\&a \Rightarrow 2016$ (outside)

(iii) $\&\&a \Rightarrow$ value at (memory
location
2016) $\Rightarrow 10$

~~$\text{printf}('/d', \&\&\&\&\&a),$~~

Why arrays ?

```
int m1, m2, m3;
```

```
float avg;
```

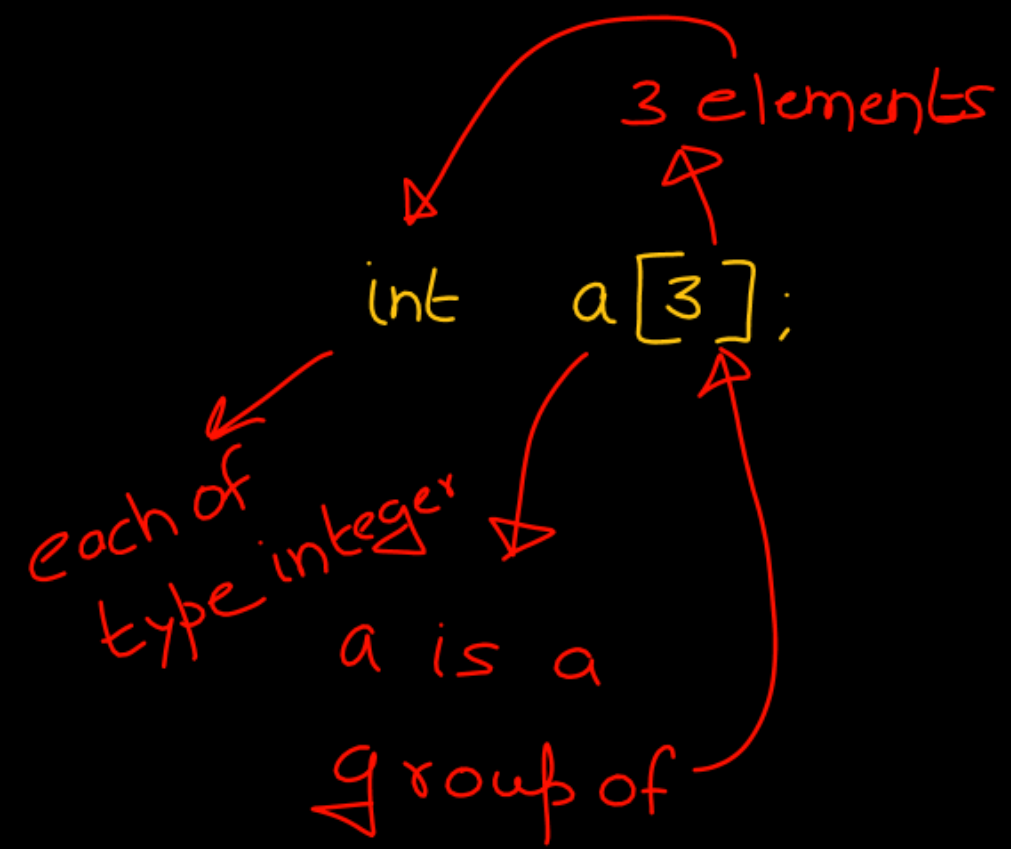
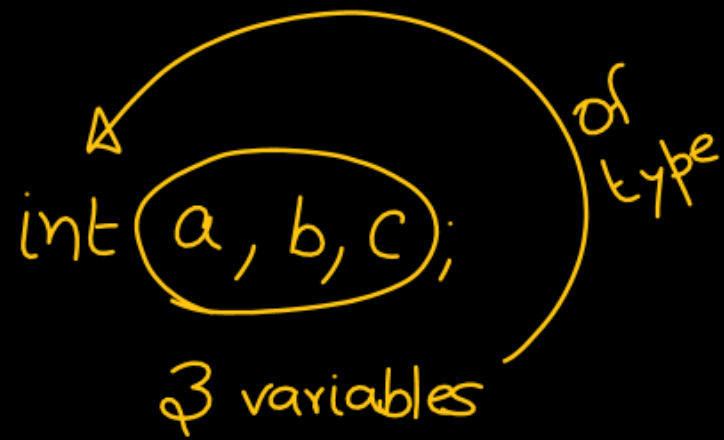
```
=====
```

```
avg = (m1 + m2 + m3) / 3.0
```

50 students

←

```
int { m1, m2, m3, m4, m5, m6, m7, m8,  
      m9, m10, . . . }
```

```
int a[50];
```

```
float b[4];
```

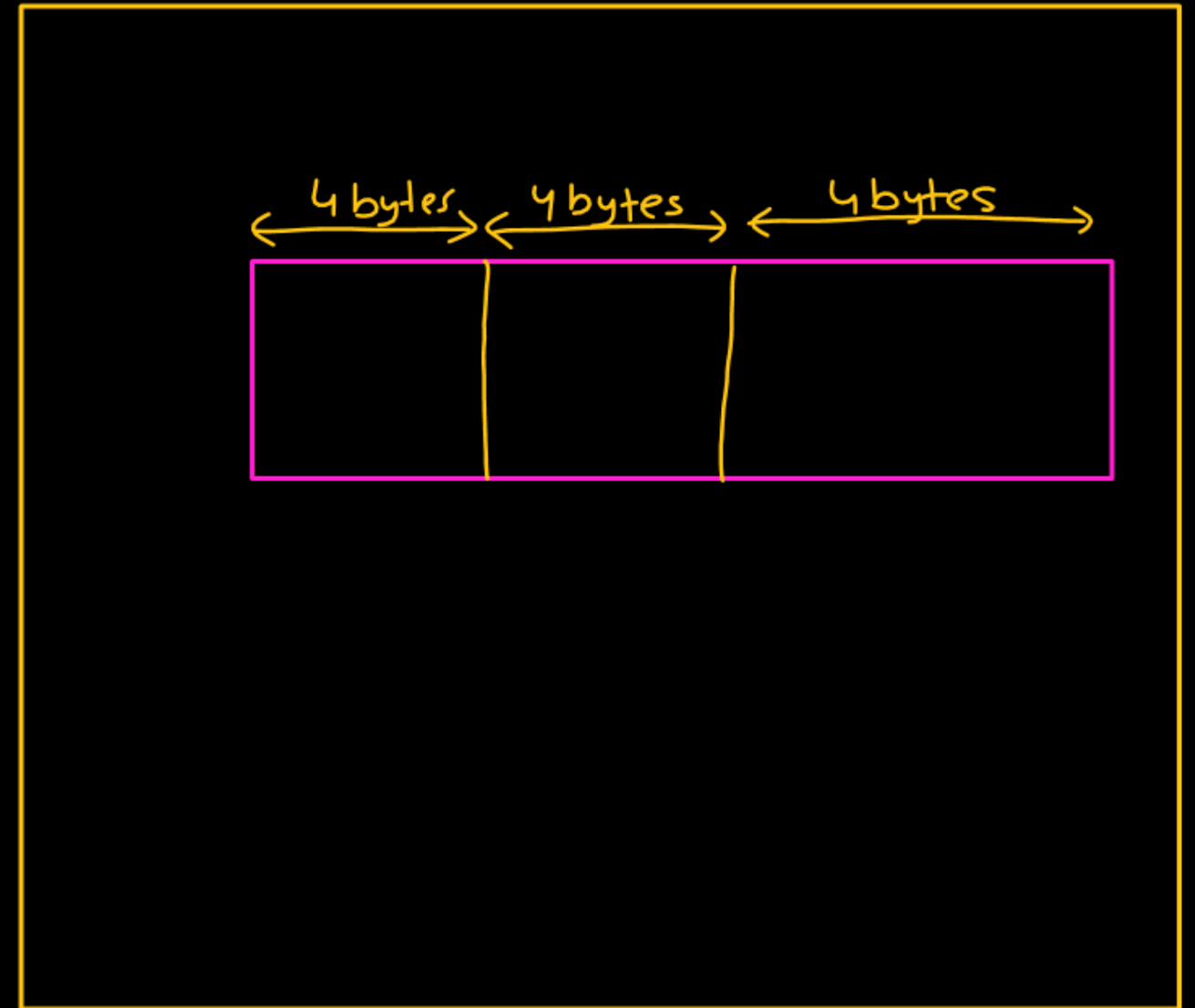
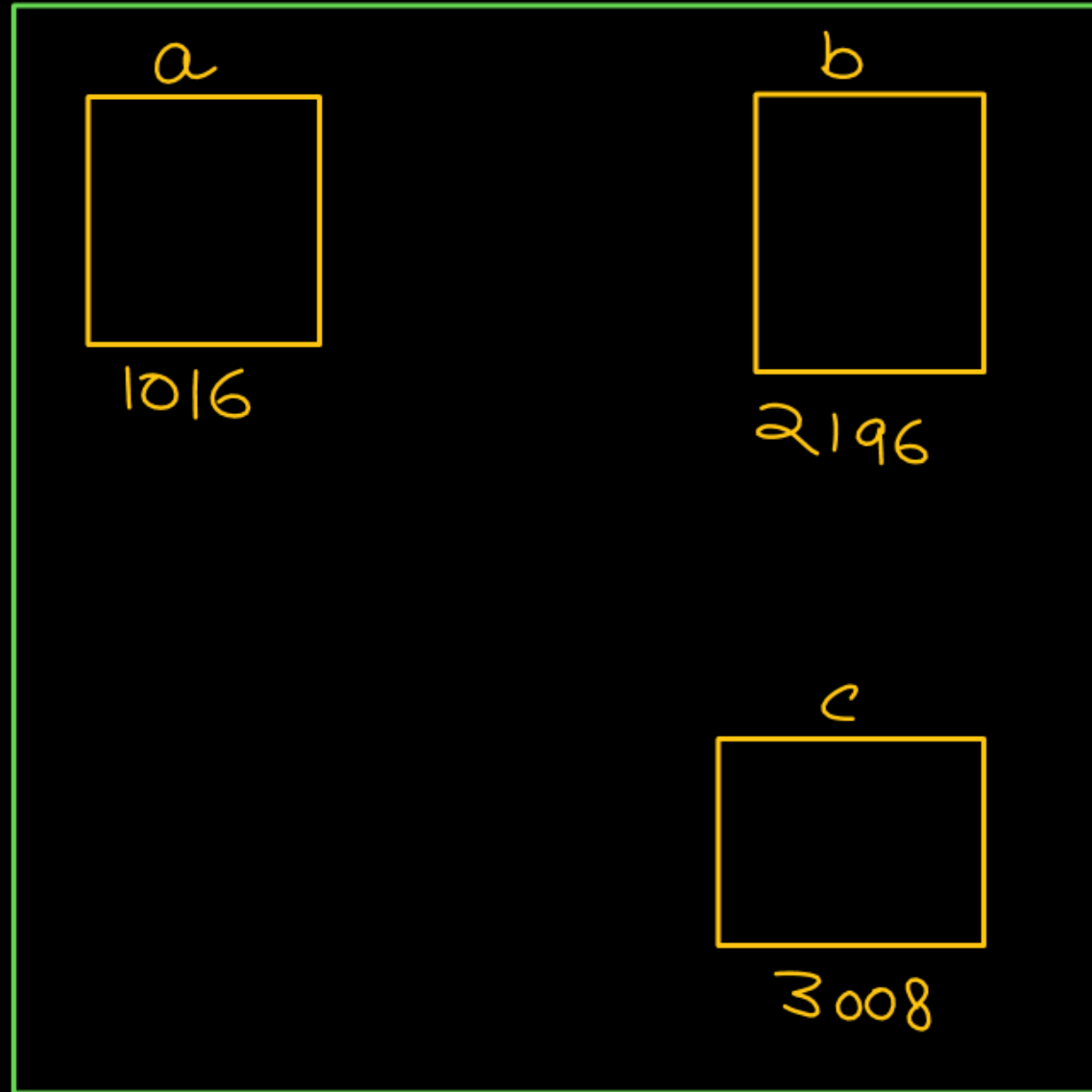
① Collection of Homogen. types of elements.

② Elements are stored seq.
one after another.

→ ③

```
int a, b, c;
```

```
int a[3]
```



```
int a, b, c;
```

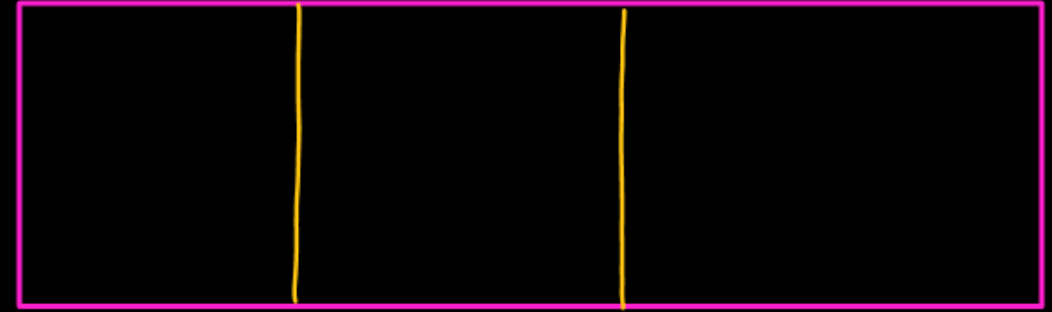
a = 10; ✓

b = 20; ✓

c = 100; ✓

```
int a[3];
```

a



~ All elements are rep.
by same name

Sec-H

group of
60 student

unique identification no.

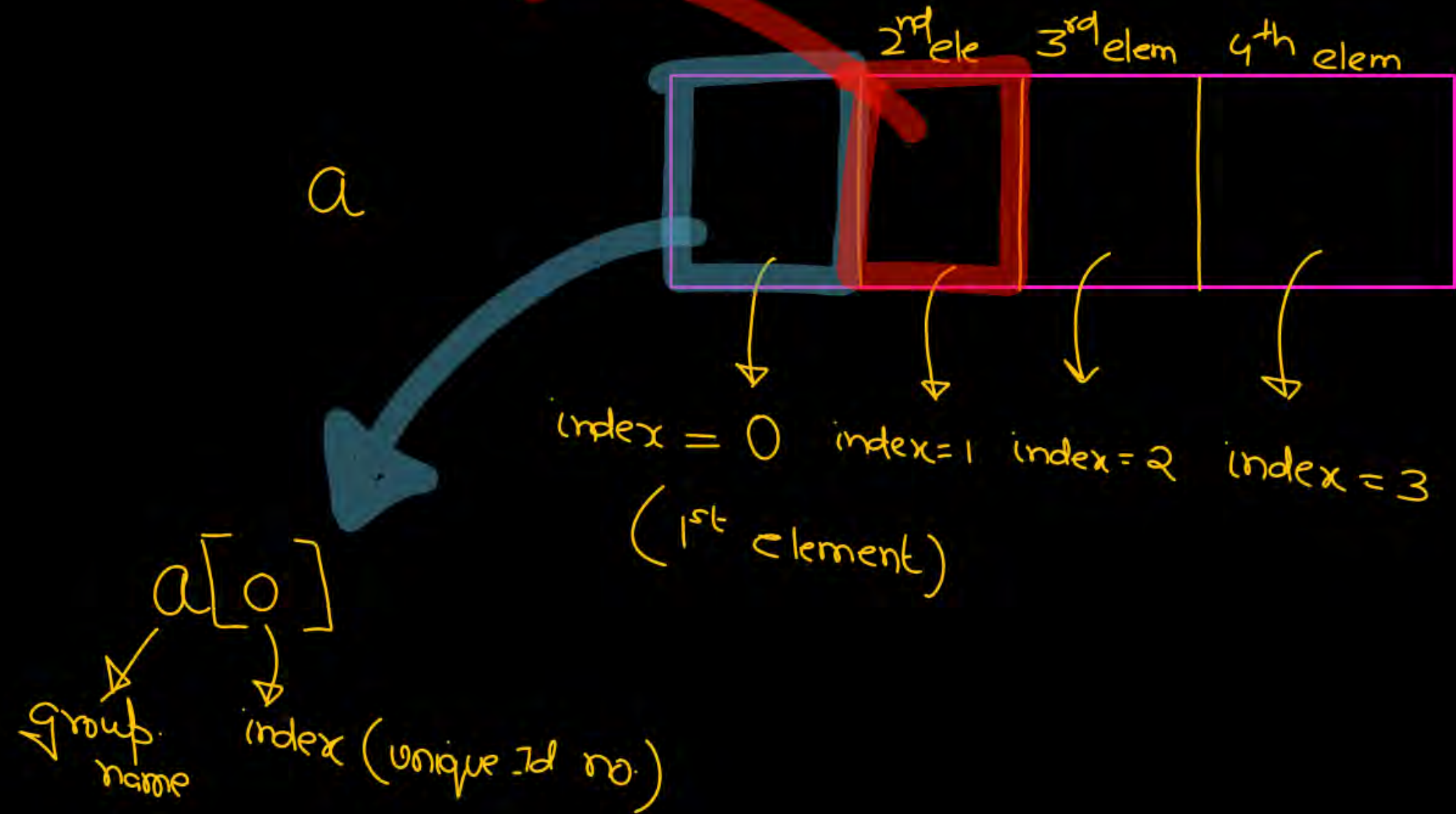
↳ Roll no.

Sec-A

group of
60 students

int a[4];

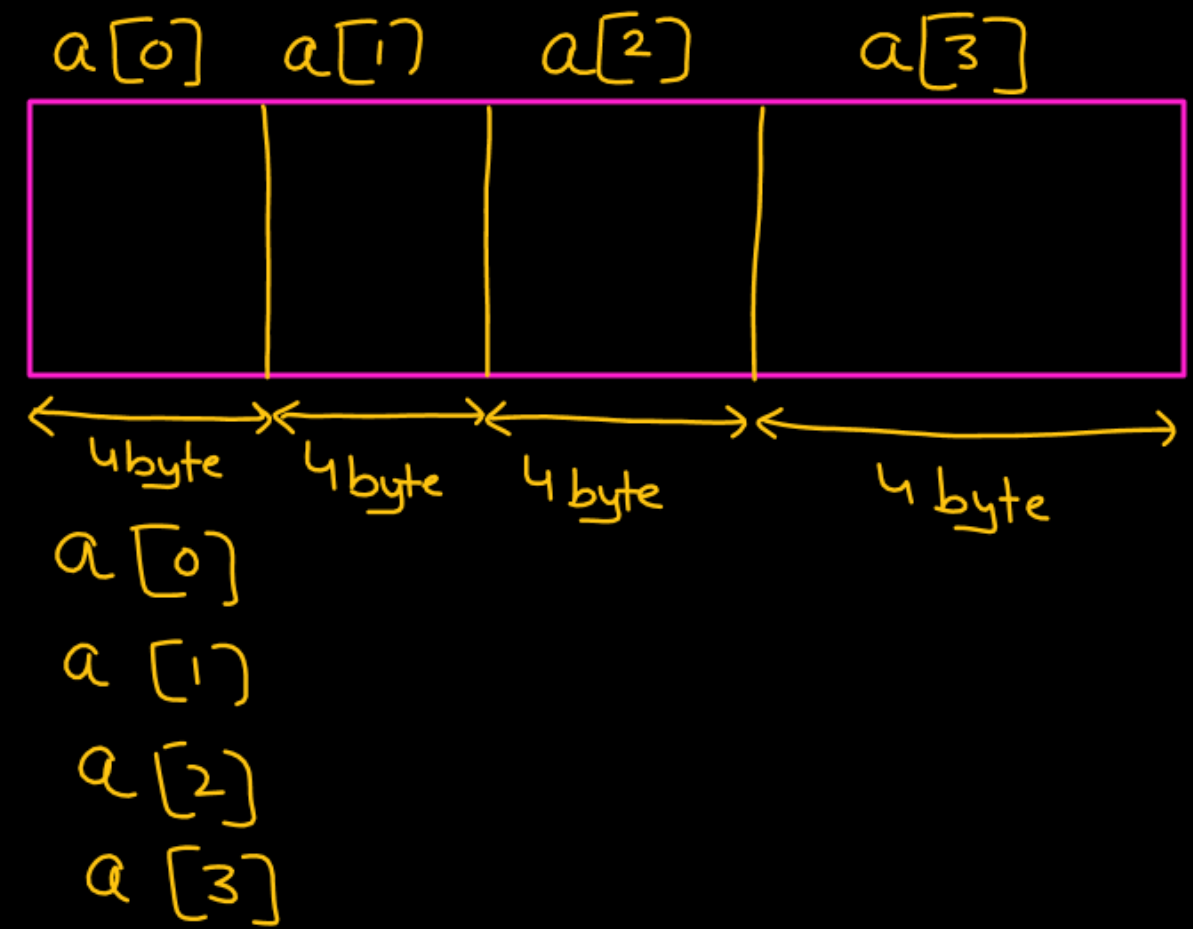
a[1]



```
int a, b, c, d;
```

a
b
c
d

```
int a[4];
```



* #
void main(){

int a;

printf("/d",a); *Garbage*
}

void main(){

int a[4];

printf("/d",a[0]); *Garbage*
printf("/d",a[1]); *Garbage*
printf("/d",a[2]); *Garbage*
printf("/d",a[3]); *Garbage*
}

1.

int a[4] = {10, 20, 30, 40};

Valid



a[0] a[1] a[2] a[3]

2.

int a[4] = {10, 20};

Valid



a[0] a[1] a[2] a[3]

3.

int a[4] = {10, 20, 30, 40, 50, 60};

Valid

ignored



a[0] a[1] a[2] a[3]

4.

int a[]; Invalid

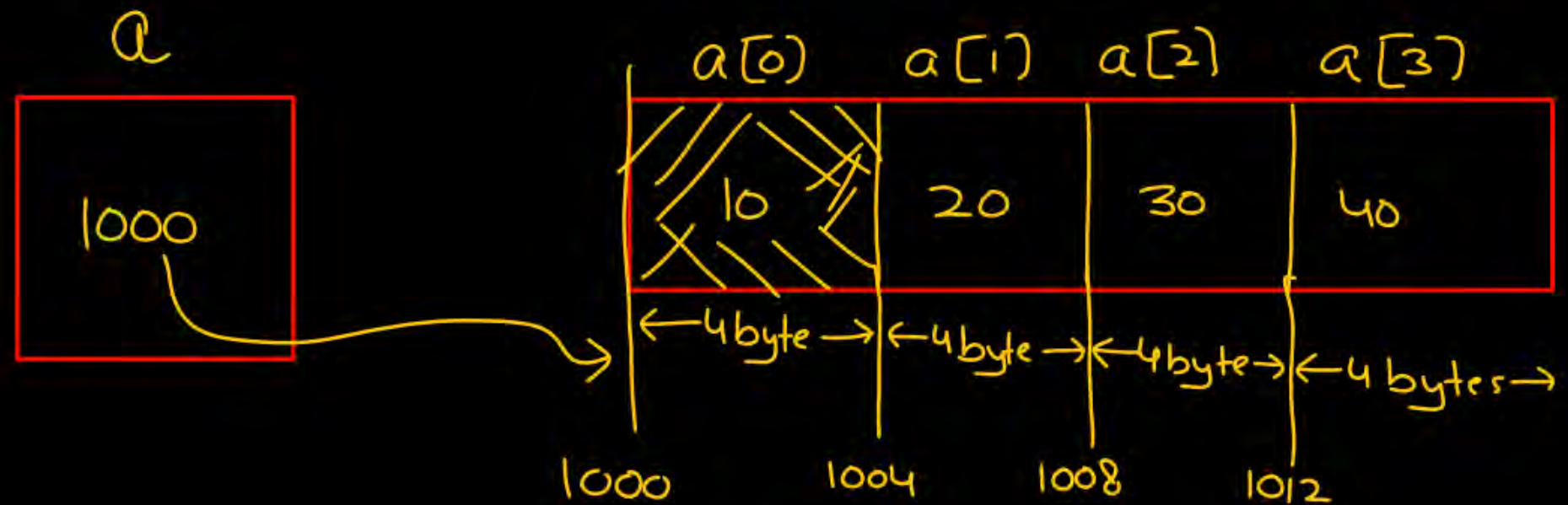
5.

int a[] = {10, 20};

Valid

2 elements

`int a[4] = {10, 20, 30, 40};`



1. Array-name holds (represent) address of its first element.
2. Array-name $\Rightarrow$ constant

1. $2 = \{ \}$ Invalid
Lvalue can not
be a constant
(literal)

Note:

Array-name = $\{ \}$

Array-name can not be L-value
for any assignment statement.

2.

2++

++2

--2;

2--;

Invalid

We can not apply
inc. dec operators
on constant.

→ 2 = 3 + 1 X

Array-name++

++Array-name

Array-name--

--Array-name

Invalid

int a = 10;

&a

a

10

2036

2036

2037

2038

2039

00000000 00000000 00000000 00001010

a : Numerical value of a is 100

$a+1$?

(1) What is a

a) simple variable or value

Arithm. $\left[\begin{array}{l} \text{value} + \text{value} = \text{value} \\ \text{int } a = 100; \\ \textcircled{a+1} \rightarrow 101 \end{array} \right.$
✓

address/pointer variable

Address arith.

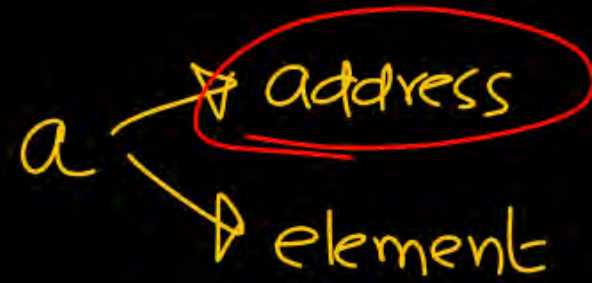
$\text{Address} + \overset{\text{Int.}}{\text{value}} \Rightarrow \text{Address}$

$\overset{\text{Int.}}{\text{value}} + \text{Add} \Rightarrow \text{Address}$

①

int a[4];

1-dimension



0-dimension

②

int a[2][3] = {1, 2, 3, 4, 5, 6};

2-dimension

a → Address/element

a[0] → Address/element

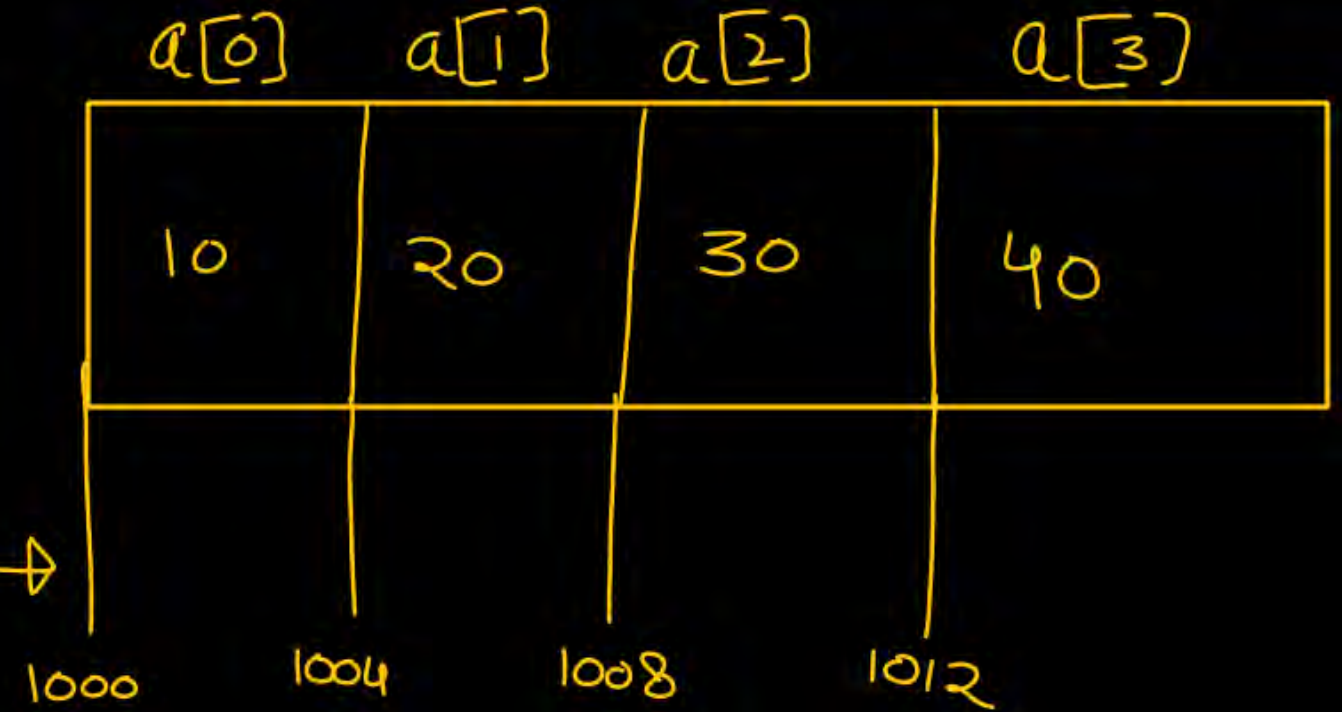
a[1] → Address/element

$a+1$?

int $a[4] = \{10, 20, 30, 40\};$

$a \equiv$ Rep. add. of
its first
element

$a \equiv \&a[0]$



$a+1$

(i) Find whether a is add/element.

→ (ii) if it is an address →

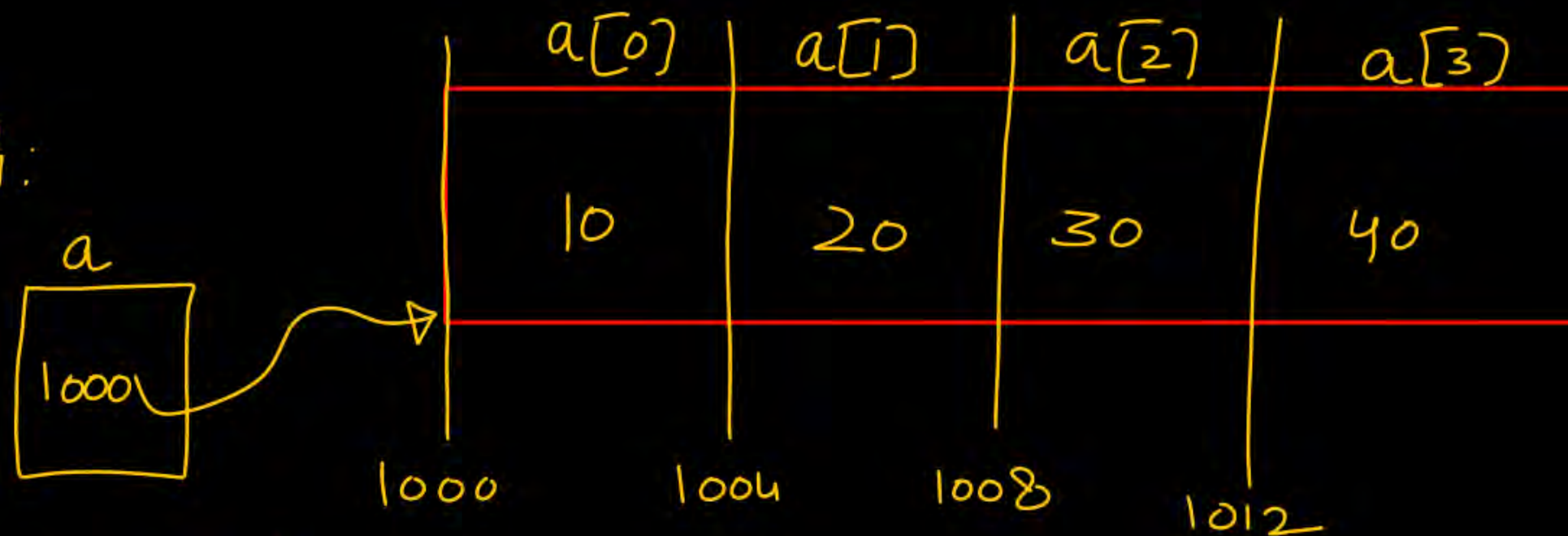
(iii) find the size of that object.

Whose address(object)
it is representing

int a[4] = {10, 20, 30, 40};

printf("/u", a);

printf("/u", &a[0]);



(1) $a \rightarrow \text{address}^{\checkmark}/\text{element}^{\times}$

(2) $a \equiv \&a[0]$

(3) $\text{sizeof } a[0] \Rightarrow (4) \text{ bytes}$

$a+1$

$\rightarrow \&a[0] + 1 \times 4$

$(a+1) \Rightarrow 1000 + 4$
 $\Rightarrow 1004$

int a[4] = {10, 20, 30, 40};

printf("/u", a);

printf("/u", &a[0]);



a[0]	a[1]	a[2]	a[3]
10	20	30	40
1000	1004	1008	1012

a+1 $\Rightarrow$ (i) a $\begin{cases} \nearrow \text{Add} \checkmark \\ \searrow \text{element} \times \end{cases}$

(ii) a $\equiv$ &a[0]

(iii) size of a[0] $\Rightarrow$ 4 bytes

$$a+1 \Rightarrow \&a[0] + 1 \times 4$$

$$= 1000 + 4$$

$$(a+1) = \text{Memory location } 1004 = \&a[1] \quad \text{---(1)}$$

a+2 $\Rightarrow$ (i) a $\begin{cases} \nearrow \text{add} \checkmark \\ \searrow \text{element} \end{cases}$ (ii) a $\equiv$ &a[0]

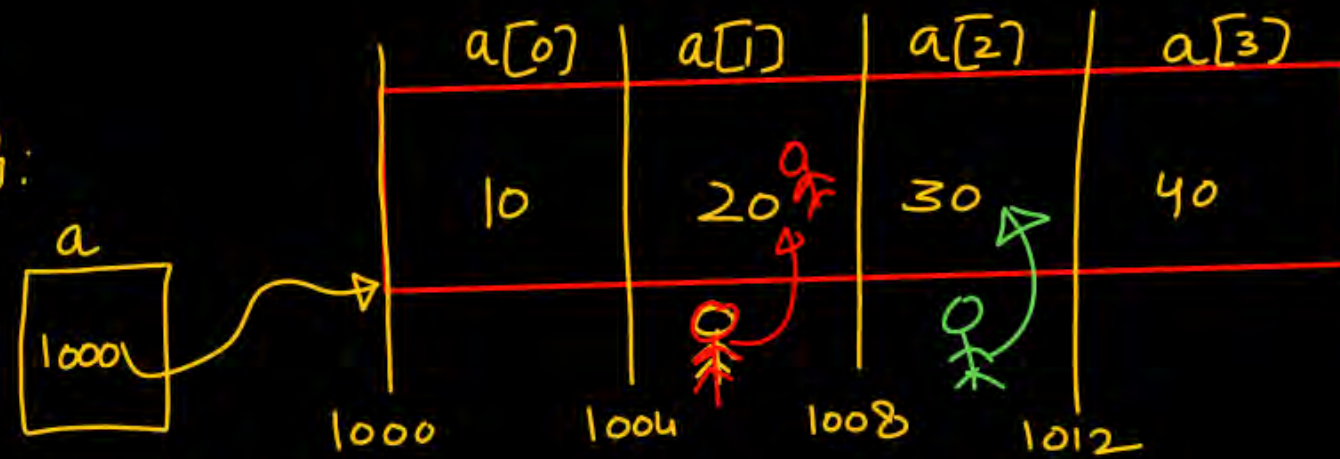
(iii) size of a[0] $\Rightarrow$ 4 bytes

$$a+2 = \&a[0] + 2 \times 4$$

$$= 1000 + 8$$

$$a+2 = \text{Memory location } 1008 = \&a[2] \quad \text{---(2)}$$

int a[4] = {10, 20, 30, 40};



$$(a+1) = \text{Memory location } 1004 = \&a[1] \quad (1)$$

$$a+2 = \text{Memory location } 1008 = \&a[2] \quad (2)$$

$$*(a+1) = \text{value at (Memory location } 1004) = \cancel{\&a[1]}$$

$$*(a+1) = 20 = a[1] \quad (3)$$

$$*(a+2) = \text{value at (Memory location } 1008) = \cancel{\&a[2]}$$

$$*(a+2) = 30 = a[2] \quad (4)$$

$$\left. \begin{array}{l} *(a+1) = a[1] \\ *(a+2) = a[2] \end{array} \right\} \text{Result}$$

$$*(a+i) = a[i]$$

```
int a[4] = {10, 20, 30, 40};
```

```
printf("%d", a[2]); 30
```

```
printf("%d", *(a+2)); 30
```


$$\checkmark * (a+i) = a[i]$$

Addition is commutative

$$a[i] = *(a+i) = *(i+a) = i[a]$$

int a[4] = {10, 20, 30, 40};

30 {
printf("%d", a[2]);
printf("%d", 2[a]);
printf("%d", *(a+2));
printf("%d", *(2+a));

int (4)[a] = {10, 20, 30, 40};

↓

Save

data_type Identifier[size]

↓

Array-name does not represent
an address with 2 operators



$\&a[0] + 1 \times 4$

```
int a[4] = {10, 20, 30, 40};
```

```
printf("/u", a+1); 1004
```

```
printf("/u", &a[0]); 1000
```

```
printf("/u", &a);
```

